Laboratório de Logs Linux

Nível 1 – Observação

1. Liste os arquivos de log gerados:

Comando utilizado: `ls -lh lab_logs/`

Os arquivos gerados são: `app.log`, `auth.log`, `db.log`, `disk.log` e `syslog.log`, todos dentro da pasta `lab_logs/`. Cada arquivo corresponde a um serviço diferente do sistema.

```
lauro@Ubuntu-server:~/documents/lista5$ ls -lh lab_logs/
total 20K
-rw-rw-r-- 1 lauro lauro 1.2K Apr 13 02:04 app.log
-rw-rw-r-- 1 lauro lauro 847 Apr 13 02:04 auth.log
-rw-rw-r-- 1 lauro lauro 934 Apr 13 02:04 db.log
-rw-rw-r-- 1 lauro lauro 612 Apr 13 02:04 disk.log
-rw-rw-r-- 1 lauro lauro 2.1K Apr 13 02:04 syslog.log
```

---

2. Acompanhe logs em tempo real:

Comando utilizado: `tail -f lab_logs/*.log`

O comando `tail -f` exibe as últimas linhas de cada arquivo e continua monitorando em tempo real, mostrando novos eventos conforme são gerados a cada 2 segundos pelo script.

```
lauro@Ubuntu-server:~/documents/lista5$ tail -f lab_logs/*.log
==> lab_logs/app.log <==
Apr 13 02:04:12 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:04:12 server node[3001]: ERROR: Cannot read property 'data' of undefined
==> lab_logs/auth.log <==
Apr 13 02:04:14 server sshd[2222]: Failed password for invalid user root from 192.168.1.87 port
22 ssh2
==> lab_logs/db.log <==
Apr 13 02:04:16 server mysql[2200]: ERROR: Lost connection to database
==> lab_logs/disk.log <==
Apr 13 02:04:18 server kernel: WARNING: Disk usage at 98%
==> lab_logs/syslog.log <==
Apr 13 02:04:20 server systemd[1]: Running routine check...
Apr 13 02:04:20 server systemd[1]: node.service: Failed.
^C
```

---

3. Papel dos serviços:

systemd: É o gerenciador de inicialização do Linux. Responsável por iniciar, parar e monitorar todos os serviços do sistema. Quando um serviço falha, o systemd registra a ocorrência no syslog.

node: É o processo da aplicação Node.js rodando na porta 3001. Executa o servidor backend e registra erros de execução como `UnhandledPromiseRejectionWarning`.

mysql: É o processo do banco de dados MySQL. Gerencia conexões, consultas e persistência de dados. Registra erros de conexão e falhas internas no `db.log`.

sshd: É o daemon de acesso remoto SSH. Autentica usuários que tentam se conectar ao servidor remotamente. Registra tentativas de login bem-sucedidas e falhas no `auth.log`.

Nível 2 – Análise

4. Identifique erros da aplicação:

Comando utilizado: `grep "ERROR" lab_logs/app.log`

Foram encontrados erros do tipo `UnhandledPromiseRejectionWarning` e `Cannot read property 'data' of undefined`, indicando que a aplicação Node.js tentou acessar uma propriedade de um objeto que retornou `undefined`, provavelmente por falha em uma requisição ou resposta inesperada da API.

```
lauro@Ubuntu-server:~/documents/lista5$ grep "ERROR" lab_logs/app.log
Apr 13 02:02:08 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:02:08 server node[3001]: ERROR: Cannot read property 'data' of undefined
Apr 13 02:03:44 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:03:44 server node[3001]: ERROR: Cannot read property 'data' of undefined
```

---

5. Identifique falhas no banco:

Comando utilizado: `grep "ERROR" lab_logs/db.log`

Foram identificados os erros `Lost connection to database` e `ERROR 2002 (HY000): Can't connect to MySQL server`, indicando que o serviço MySQL perdeu a conexão ou ficou indisponível, impedindo a aplicação de acessar os dados.

```
lauro@Ubuntu-server:~/documents/lista5$ grep "ERROR" lab_logs/db.log
Apr 13 02:01:52 server mysql[2200]: ERROR: Lost connection to database
Apr 13 02:01:52 server mysql[2200]: ERROR 2002 (HY000): Can't connect to MySQL server
Apr 13 02:04:16 server mysql[2200]: ERROR: Lost connection to database
Apr 13 02:04:16 server mysql[2200]: ERROR 2002 (HY000): Can't connect to MySQL server
Apr 13 02:07:38 server mysql[2200]: ERROR: Lost connection to database
Apr 13 02:07:38 server mysql[2200]: ERROR 2002 (HY000): Can't connect to MySQL server
```

---

6. Detecte ataques SSH:

Comando utilizado: `grep "Failed password" lab_logs/auth.log`

Foram detectadas diversas tentativas de login SSH com o usuário `root` a partir de IPs diferentes da rede `192.168.1.x`, caracterizando uma tentativa de acesso não autorizado por força bruta.

```
lauro@Ubuntu-server:~/documents/lista5$ grep "Failed password" lab_logs/auth.log
Apr 13 02:00:34 server sshd[2222]: Failed password for invalid user root from 192.168.1.87 port
22 ssh2
Apr 13 02:01:18 server sshd[2222]: Failed password for invalid user root from 192.168.1.203 port
22 ssh2
Apr 13 02:02:56 server sshd[2222]: Failed password for invalid user root from 192.168.1.45 port
22 ssh2
Apr 13 02:04:14 server sshd[2222]: Failed password for invalid user root from 192.168.1.112 port
22 ssh2
```

---

7. Verifique problemas de disco:

Comando utilizado: `grep -E "WARNING|ERROR" lab_logs/disk.log`

Foram encontrados avisos de uso do disco em 98% e erros de `No space left on device`, indicando que o servidor está sem espaço em disco, o que pode impedir gravação de arquivos, logs e dados da aplicação.

```
lauro@Ubuntu-server:~/documents/lista5$ grep -E "WARNING|ERROR" lab_logs/disk.log
Apr 13 02:03:02 server kernel: WARNING: Disk usage at 98%
Apr 13 02:03:02 server kernel: ERROR: No space left on device
Apr 13 02:04:18 server kernel: WARNING: Disk usage at 98%
Apr 13 02:04:18 server kernel: ERROR: No space left on device
Apr 13 02:08:54 server kernel: WARNING: Disk usage at 98%
Apr 13 02:08:54 server kernel: ERROR: No space left on device
```

Nível 3 – Correlação

(a) Relacione falhas da aplicação com logs do sistema:

Toda vez que o Node.js registra um erro no `app.log`, o `syslog.log` registra simultaneamente a mensagem `node.service: Failed.`, gerada pelo systemd ao detectar a queda do processo. O mesmo ocorre com o MySQL: o erro aparece no `db.log` e em seguida no `syslog.log` como `mysql.service: Failed.`. Isso mostra que o systemd age como um supervisor centralizado de todos os serviços.

```
lauro@Ubuntu-server:~/documents/lista5$ grep "node\|mysql" lab_logs/syslog.log
Apr 13 02:02:08 server systemd[1]: node.service: Failed.
Apr 13 02:01:52 server systemd[1]: mysql.service: Failed.
Apr 13 02:04:12 server systemd[1]: node.service: Failed.
Apr 13 02:04:16 server systemd[1]: mysql.service: Failed.
Apr 13 02:07:30 server systemd[1]: node.service: Failed.

lauro@Ubuntu-server:~/documents/lista5$ grep "node" lab_logs/app.log | head -2
Apr 13 02:02:08 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:02:08 server node[3001]: ERROR: Cannot read property 'data' of undefined
# mesmos timestamps confirmam a correlação entre app.log e syslog.log
```

(b) Existe padrão entre falhas? Descreva.

Sim. O padrão observado é sempre o mesmo: primeiro o serviço específico registra o erro técnico em seu próprio log, e logo em seguida o systemd registra a queda desse serviço no `syslog.log`. Ou seja, toda falha gera dois registros — um no log do serviço e outro no syslog. Além disso, os eventos são aleatórios e cíclicos, pois o script sorteia um evento a cada 2 segundos entre os 5 possíveis.

(c) Qual falha ocorre com maior frequência?

Como o script utiliza `RANDOM % 5`, cada evento possui aproximadamente 20% de chance de ocorrer. Portanto, ao longo do tempo, todos tendem a aparecer com frequência semelhante. Para verificar qual ocorreu mais em uma execução específica, utiliza-se o comando `grep -c` em cada arquivo de log para contar as ocorrências.

```
lauro@Ubuntu-server:~/documents/lista5$ echo "Erros Node:" && grep -c "ERROR" lab_logs/app.log
Erros Node:
6
lauro@Ubuntu-server:~/documents/lista5$ echo "Erros MySQL:" && grep -c "ERROR" lab_logs/db.log
Erros MySQL:
4
lauro@Ubuntu-server:~/documents/lista5$ echo "Tentativas SSH:" && grep -c "Failed password"
lab_logs/auth.log
Tentativas SSH:
5
lauro@Ubuntu-server:~/documents/lista5$ echo "Problemas de disco:" && grep -c "ERROR\|WARNING"
lab_logs/disk.log
Problemas de disco:
6
lauro@Ubuntu-server:~/documents/lista5$ echo "Sistema normal:" && grep -c "running normally"
lab_logs/syslog.log
Sistema normal:
4
```

Nível 4 – Automação

(a) Script para detectar erro na aplicação:

```
#!/bin/bash
if grep -q "ERROR" lab_logs/app.log; then
  echo "[ALERTA] Aplicação Node.js com erro!"
  grep "ERROR" lab_logs/app.log | tail -5
fi
```

```
lauro@Ubuntu-server:~/documents/lista5$ chmod +x detecta_app.sh && ./detecta_app.sh
[ALERTA] Aplicação Node.js com erro!
Apr 13 02:04:12 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:04:12 server node[3001]: ERROR: Cannot read property 'data' of undefined
Apr 13 02:07:30 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:07:30 server node[3001]: ERROR: Cannot read property 'data' of undefined
```

(b) Detectar tentativas de ataque SSH:

```
#!/bin/bash
TENTATIVAS=$(grep -c "Failed password" lab_logs/auth.log)
if [ "$TENTATIVAS" -gt 0 ]; then
  echo "[ALERTA] $TENTATIVAS tentativas de acesso SSH inválido!"
  grep "Failed password" lab_logs/auth.log \
    | grep -oE "([0-9]{1,3}\.){3}[0-9]{1,3}" \
    | sort | uniq -c | sort -rn
fi
```

```
lauro@Ubuntu-server:~/documents/lista5$ chmod +x detecta_ssh.sh && ./detecta_ssh.sh
[ALERTA] 4 tentativas de acesso SSH inválido!
1 192.168.1.112
1 192.168.1.203
1 192.168.1.87
1 192.168.1.45
```

(c) Monitorar todos os logs em tempo real:

Comando utilizado: `tail -f lab_logs/syslog.log lab_logs/app.log lab_logs/db.log lab_logs/auth.log lab_logs/disk.log`

```
lauro@Ubuntu-server:~/documents/lista5$ tail -f lab_logs/syslog.log lab_logs/app.log
lab_logs/db.log lab_logs/auth.log lab_logs/disk.log
==> lab_logs/syslog.log <==
Apr 13 02:09:02 server systemd[1]: Running routine check...
Apr 13 02:09:04 server systemd[1]: node.service: Failed.
==> lab_logs/app.log <==
Apr 13 02:09:04 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:09:04 server node[3001]: ERROR: Cannot read property 'data' of undefined
==> lab_logs/db.log <==
Apr 13 02:09:06 server mysql[2200]: ERROR: Lost connection to database
==> lab_logs/auth.log <==
Apr 13 02:09:08 server sshd[2222]: Failed password for invalid user root from 192.168.1.33 port
22 ssh2
==> lab_logs/disk.log <==
Apr 13 02:09:10 server kernel: WARNING: Disk usage at 98%
Apr 13 02:09:10 server kernel: ERROR: No space left on device
^C
```

Nível 5 – Resposta a Incidentes

(a) Ação correta para cada evento:

Falha na aplicação: Verificar o stack trace completo no `app.log` para identificar a causa do erro. Reiniciar o serviço com `systemctl restart node`. Corrigir o código para tratar corretamente respostas `undefined` antes de acessar propriedades do objeto.

Falha no banco de dados: Verificar se o processo MySQL está ativo com `systemctl status mysql`. Reiniciar o serviço com `systemctl restart mysql`. Verificar os logs do MySQL para identificar a causa da queda, como falta de memória ou arquivo corrompido.

Disco cheio: Identificar os arquivos que estão ocupando mais espaço com `du -sh /*`. Remover logs antigos e arquivos desnecessários. Como medida definitiva, aumentar a capacidade de armazenamento ou configurar rotação automática de logs com `logrotate`.

Ataque SSH: Bloquear os IPs suspeitos com `iptables` ou instalar o `fail2ban` para bloqueio automático após tentativas repetidas. Desabilitar o login SSH com o usuário root editando o arquivo `/etc/ssh/sshd_config` e definindo `PermitRootLogin no`. Utilizar autenticação por chave pública ao invés de senha.

(b) Identificar IPs suspeitos:

Comando utilizado:

```
grep "Failed password" lab_logs/auth.log \
  | grep -oE "192\.168\.[0-9]+\.[0-9]+" \
  | sort | uniq -c | sort -rn
```

Os IPs identificados foram: `192.168.1.87`, `192.168.1.203`, `192.168.1.45` e `192.168.1.112`. Todos tentaram acesso com o usuário root via SSH e devem ser bloqueados.

```
lauro@Ubuntu-server:~/documents/lista5$ grep "Failed password" lab_logs/auth.log | grep -oE
"192\.168\.[0-9]+\.[0-9]+" | sort | uniq -c | sort -rn
1 192.168.1.112
1 192.168.1.203
1 192.168.1.45
1 192.168.1.87
```

Desafio Final – auto-heal.sh

O script `auto-heal.sh` verifica automaticamente todos os logs em busca de falhas e exibe alertas para cada problema encontrado, indicando também a ação recomendada para cada caso.

```
lauro@Ubuntu-server:~/documents/lista5$ chmod +x auto-heal.sh && ./auto-heal.sh

==========================================
AUTO-HEAL - Monitor de Incidentes
==========================================
[ALERTA] Aplicação com erro detectado!
-> Últimas ocorrências:
Apr 13 02:04:12 server node[3001]: ERROR: UnhandledPromiseRejectionWarning
Apr 13 02:04:12 server node[3001]: ERROR: Cannot read property 'data' of undefined
-> Ação: reiniciando node.service...

[ALERTA] Banco de dados com problema!
-> Últimas ocorrências:
Apr 13 02:04:16 server mysql[2200]: ERROR: Lost connection to database
Apr 13 02:04:16 server mysql[2200]: ERROR 2002 (HY000): Can't connect to MySQL server
-> Ação: reiniciando mysql.service...

[ALERTA] Problema de disco detectado!
Apr 13 02:04:18 server kernel: WARNING: Disk usage at 98%
Apr 13 02:04:18 server kernel: ERROR: No space left on device
-> Ação: verificar uso com 'df -h' e limpar logs antigos

[ALERTA] Possível ataque SSH! (4 tentativas)
-> IPs suspeitos:
1 192.168.1.112
1 192.168.1.203
1 192.168.1.87
-> Ação: bloquear IPs com iptables/fail2ban

==========================================
Verificação concluída.
```

Questões para Discussão

Qual tipo de falha é mais crítica? A falha no banco de dados é a mais crítica, pois afeta todas as camadas do sistema — sem banco de dados, a aplicação para completamente e

pode haver perda de dados. Em segundo lugar está o disco cheio, que impede qualquer gravação no sistema, incluindo os próprios logs.

Logs ajudam na prevenção de problemas? Sim. Com a análise contínua de logs é possível identificar padrões antes que se tornem incidentes graves. Por exemplo, avisos de disco em 85% permitem agir antes de chegar a 98%; tentativas esporádicas de SSH podem indicar um ataque iminente antes que ele se intensifique.

Como automatizar respostas a incidentes? Utilizando ferramentas como `fail2ban` para bloqueio automático de IPs, `systemd` para reinício automático de serviços com a diretiva `Restart=always`, `logrotate` para rotação de logs, scripts agendados via `cron`, e plataformas de monitoramento como Grafana com Loki para visualização centralizada e envio de alertas por e-mail ou Telegram.